Universidad Autónoma de Tamaulipas
Unidad Académica Multidisciplinaria Mante Centro

Nombre del trabajo:
Laboratorios 05 - 08

Asignatura:
Arquitectura de Equipos Digitales

Alumnos:
Martínez Álvarez Ángel Enrique
Martínez Ontiveros Luis Fernando
Mejía Escobar Lizeth

VERDAD, BELLEZA, PROBIDAD

Profesor:
López Piña Daniel

Grado: 6° **Grupo:** J

Ciudad Mante, Tamaulipas abril de 2026

Índice

Evidencias 3

Laboratorio 05. Comunicación serial UART: Interfaz entre MCU y PC. 3

Laboratorio 06. Sensor I2C: Lectura de Temperatura y Humedad con BMP280 / HDT. 8

Laboratorio 07. PWM y Control Analogico: Dimmer de LED y Control Servo. 16

Laboratorio 08. SPI en Acción: Comunicación con pantalla TFT ILI9341. 19

VERDAD, BELLEZA, PROBIDAD

Evidencias

Laboratorio 05. Comunicación serial UART: Interfaz entre MCU y PC.

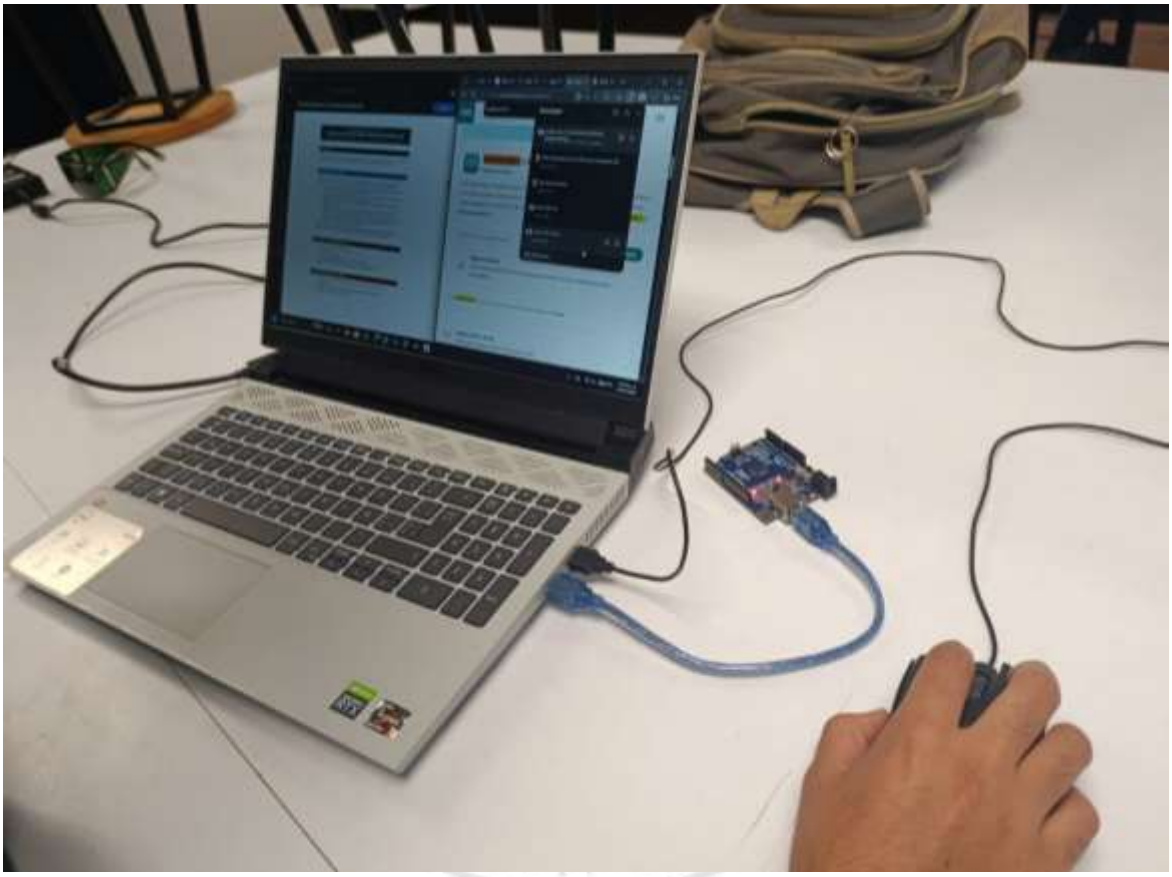

Imagen 1.
Conexión física del Arduino UNO a la computadora mediante cable USB para comunicación serial UART. Se observa el IDE de Arduino abierto en la pantalla del equipo, listo para programar y monitorear la transmisión de datos.

Imagen 2.

Código fuente del sketch UART en el IDE de Arduino. Se aprecia la lógica de procesamiento de comandos seriales (LED_ON, LED_OFF, STATUS, BLINK) y en la ventana de salida se muestra la compilación exitosa con el uso de memoria del programa.

VERDAD, BELLEZA, PROBIDAD

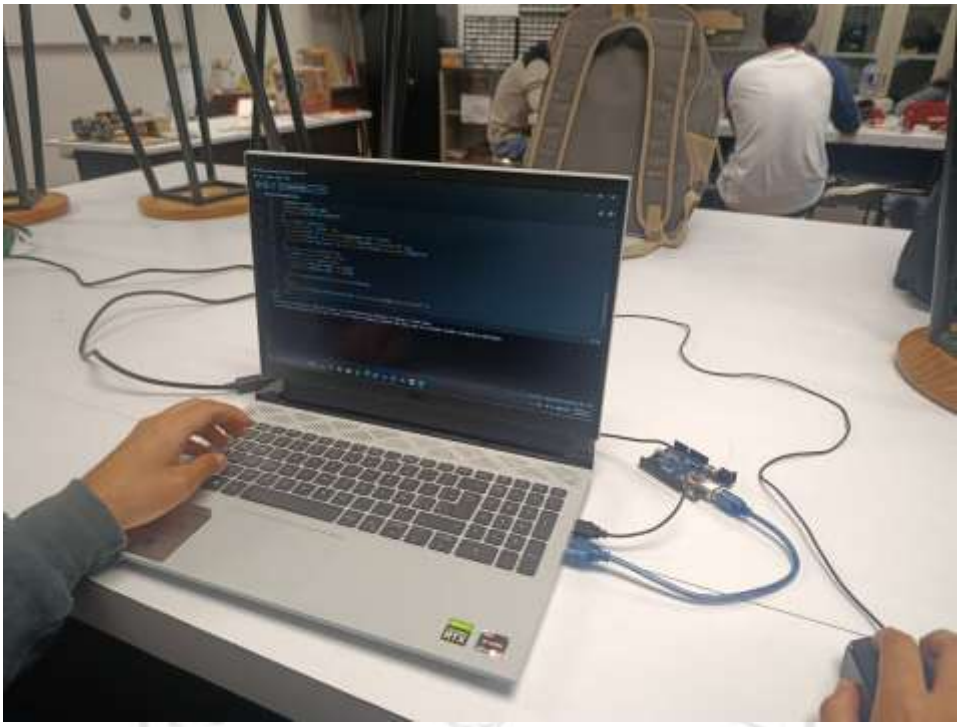

Imagen 3.
Vista general de la estación de trabajo durante la práctica de comunicación serial UART. El alumno interactúa con el teclado mientras el Arduino UNO permanece conectado y ejecutando el firmware de comandos seriales en tiempo real.

Preguntas de reflexión y reporte.

¿Cuántos bits se transmiten por carácter en formato 8N1 a 115200 bps?

- El formato 8N1 significa:
- 1 bit de Start
- 8 bits de datos
- N = sin paridad (0 bits)
- 1 bit de Stop
- Total = 10 bits por carácter

¿Cuánto tiempo tarda en transmitirse la cadena 'LED_ON' completa?

LED_ON tiene 6 caracteres: 6 × 10 bits = 60 bits
Tiempo = 60 bits ÷ 115200 bps = 0.000520 segundos = 0.52 ms
A 115200 bps es extremadamente rápido. A 9600 bps tardaría 6.25 ms, unas 12 veces más lento.

¿Qué ocurre si el Arduino está configurado a 115200 bps pero el monitor serial está a 9600 bps? Prueba este escenario y describe el comportamiento.

VERDAD, BELLEZA, PROBIDAD

Caracteres ilegibles, símbolos extraños, ningún texto coherente. El Arduino al envía los bits a 115200 bps, y el monitor los interpreta a 9600 bps, así que cada bit se lee en el momento equivocado, lo que genera caracteres incorrectos.

¿Por qué se usa un buffer de String para acumular los caracteres del comando en lugar de procesar cada byte individualmente al recibirlo?

Porque los comandos llegan carácter por carácter, no todos juntos, el buffer acumula todo hasta recibir el '\n' (Enter), y solo entonces procesa el comando completo. Sin el buffer, sería imposible distinguir un comando de otro.

El comando 'BLINK:N' usa delay() dentro del procesador de comandos. ¿Qué problema tiene esto? ¿Como rediseñarías el sistema para que el Arduino pueda recibir otros comandos mientras parpadea?

El delay() bloquea completamente el programa. Mientras el Arduino ejecuta esto:

```
for (int i = 0; i < veces; i++) {
   digitalWrite(LED_PIN, HIGH); delay(150);  // bloqueado 150ms
   digitalWrite(LED_PIN, LOW);  delay(150);  // bloqueado 150ms
}
```

Durante ese tiempo no puede hacer nada más: no lee el puerto serial, no procesa nuevos comandos, no actualiza ningún sensor. Si se manda el LED_OFF mientras parpadea, el Arduino no lo recibirá hasta que termine el loop completo.

¿Como extenderías el protocolo para que el Arduino envíe datos de sensor cada 2 segundos Y al mismo tiempo procese comandos entrantes? Describe la arquitectura del firmware.

```
// Variables globales
bool parpadeando = false;
int vecesRestantes = 0;
bool estadoBlink = false;
unsigned long ultimoBlink = 0;
const int INTERVALO_BLINK = 150;

void loop() {
  // Leer comandos normalmente (siempre activo)
  while (Serial.available() > 0) {
    char c = Serial.read();
    if (c == '\n') { procesarComando(bufferCmd); bufferCmd = ""; }
    else if (c != '\r') bufferCmd += c;
  }

  // Manejar parpadeo
  if (parpadeando && millis() - ultimoBlink >= INTERVALO_BLINK) {
    ultimoBlink = millis();
```

```
    estadoBlink = !estadoBlink;
    digitalWrite(LED_PIN, estadoBlink ? HIGH : LOW);
    if (!estadoBlink) vecesRestantes--; // cuenta al apagar
    if (vecesRestantes <= 0) {
      parpadeando = false;
      digitalWrite(LED_PIN, LOW);
      Serial.println("[OK] Blink completado");
    }
  }
}

void loop() {

  // TAREA 1: Leer comandos entrantes
  while (Serial.available() > 0) {
    char c = Serial.read();
    if (c == '\n') { procesarComando(bufferCmd); bufferCmd = ""; }
    else if (c != '\r') bufferCmd += c;
  }

  // TAREA 2: Enviar datos del sensor cada 2 segundos
  if (millis() - ultimoSensor >= INTERVALO_SENSOR) {
    ultimoSensor = millis();
    // Aquí iría la lectura real del sensor
    int valorSensor = analogRead(A0);
    Serial.print("[SENSOR] Valor: ");
    Serial.println(valorSensor);
  }
}
```

Descripción de la arquitectura.
loop() se ejecuta miles de veces por segundo
Este patrón se llama "multitarea cooperativa" y es la base del firmware profesional en microcontroladores. Ninguna tarea bloquea a las demás porque ninguna usa delay().

Universidad Autónoma de TAMAULIPAS

Unidad Académica Multidisciplinaria Mante

Laboratorio 06. Sensor I2C: Lectura de Temperatura y Humedad con BMP280 / HDT.

VERDAD, BELLEZA, PROBIDAD

Aniversario 75 UAT 1950-2025

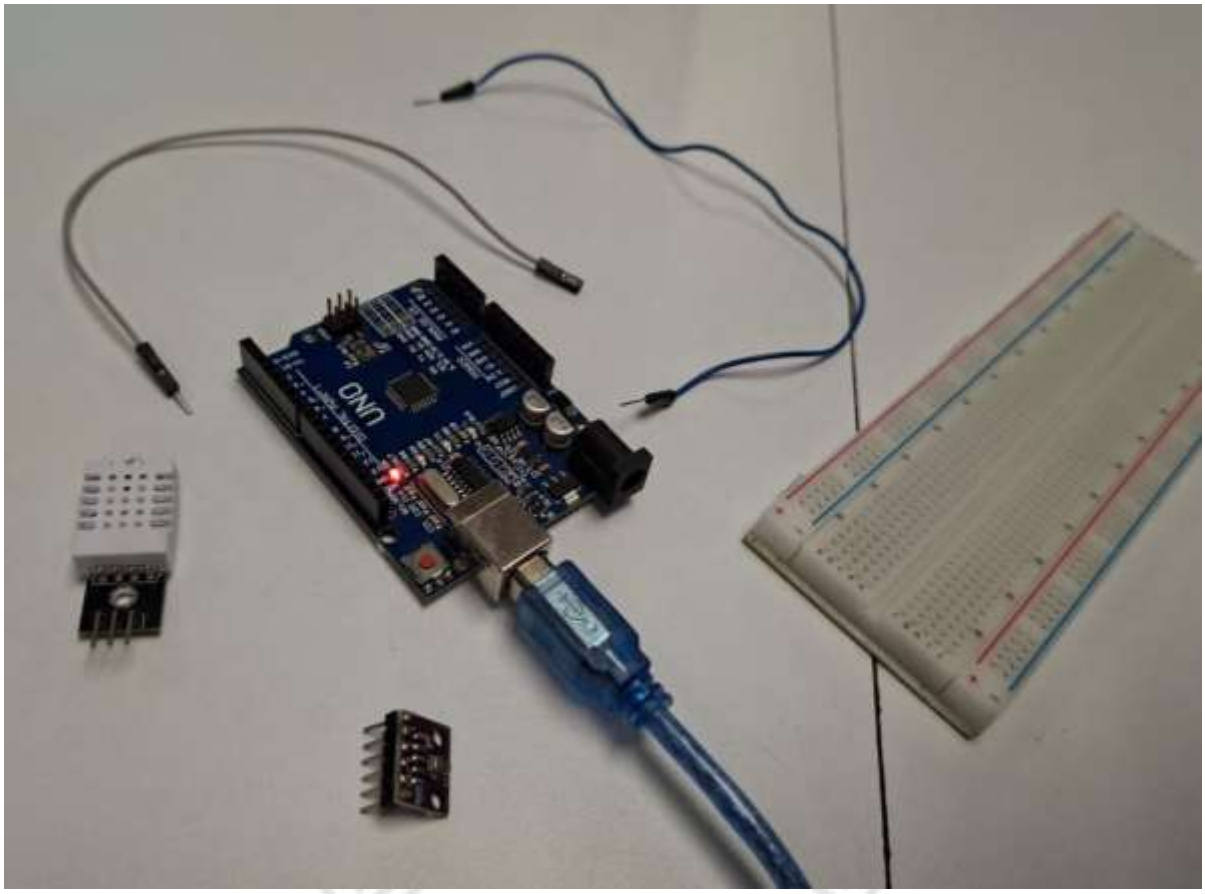

Imagen 4.
Componentes del Laboratorio 06: Arduino UNO, sensor DHT22 de temperatura y humedad, módulo BMP280 (presión/altitud), protoboard y cables de conexión. Todos los elementos dispuestos antes del armado del circuito I2C.

VERDAD, BELLEZA, PROBIDAD

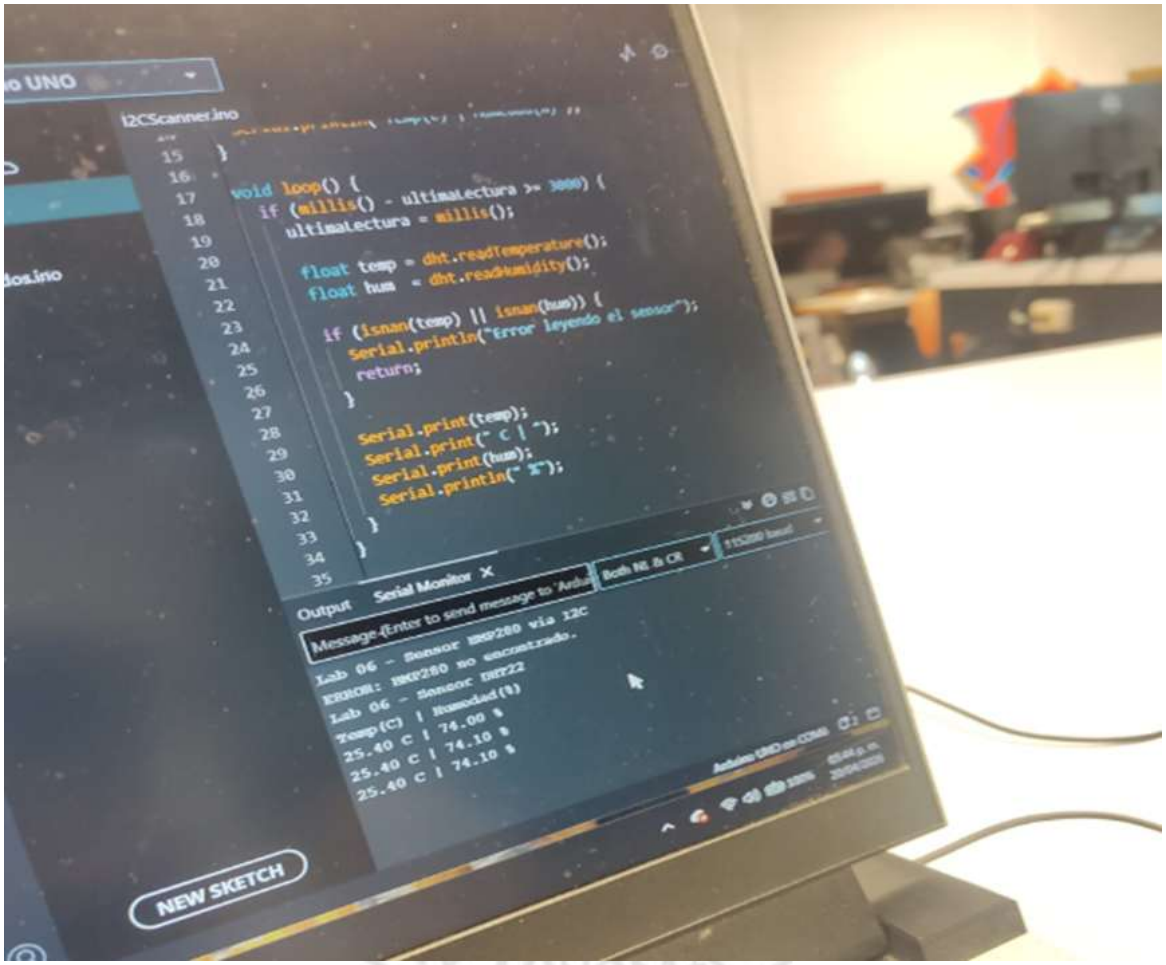

Imagen 5.
Monitor Serial mostrando las lecturas del sensor DHT22 mediante el protocolo I2C: temperatura de 25.40°C y humedad del 74.10%. Se observa también el código fuente con la lógica de lectura periódica usando millis() y el resultado en consola.

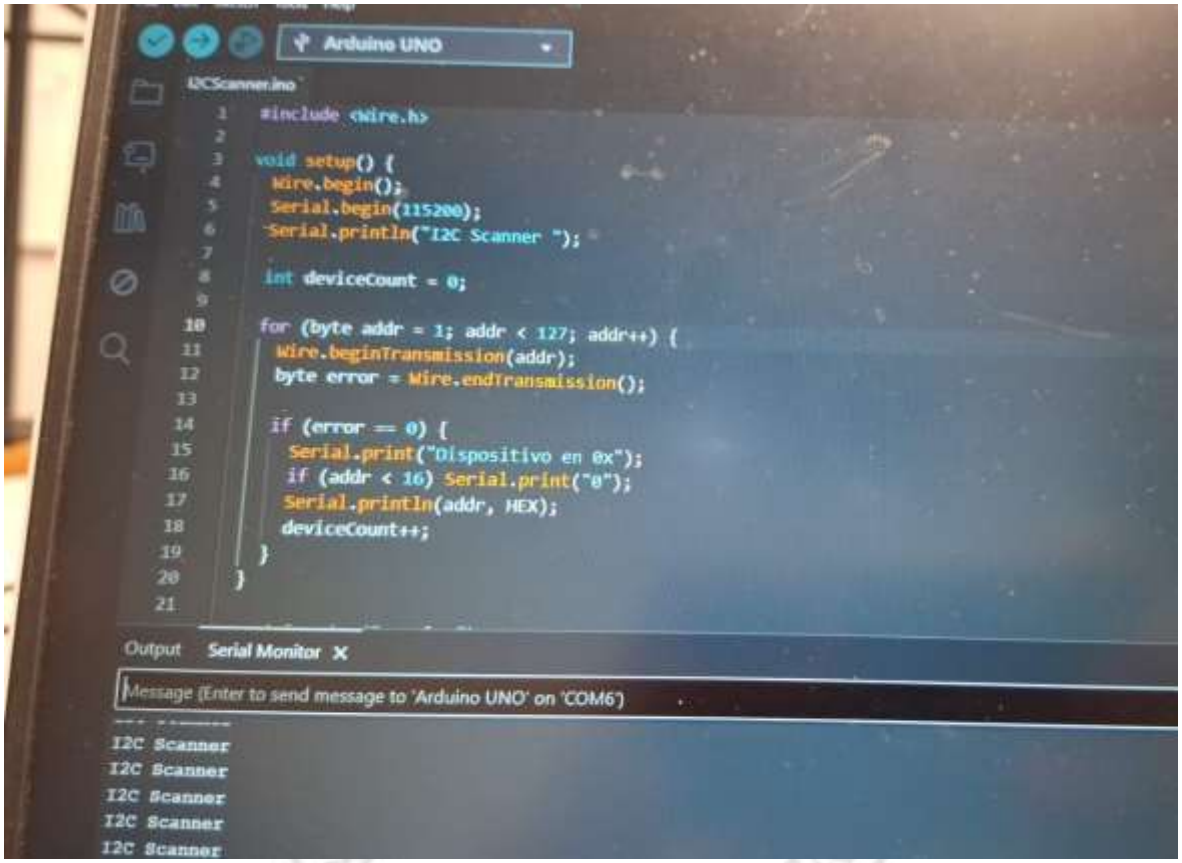

Imagen 6.

Ejecución del sketch I2C Scanner en Arduino IDE. El código escanea las 127 direcciones del bus I2C y el Monitor Serial reporta los dispositivos detectados. Este paso confirma que el sensor está correctamente conectado y responde en su dirección asignada (0x76 o 0x77).

VERDAD, BELLEZA, PROBIDAD

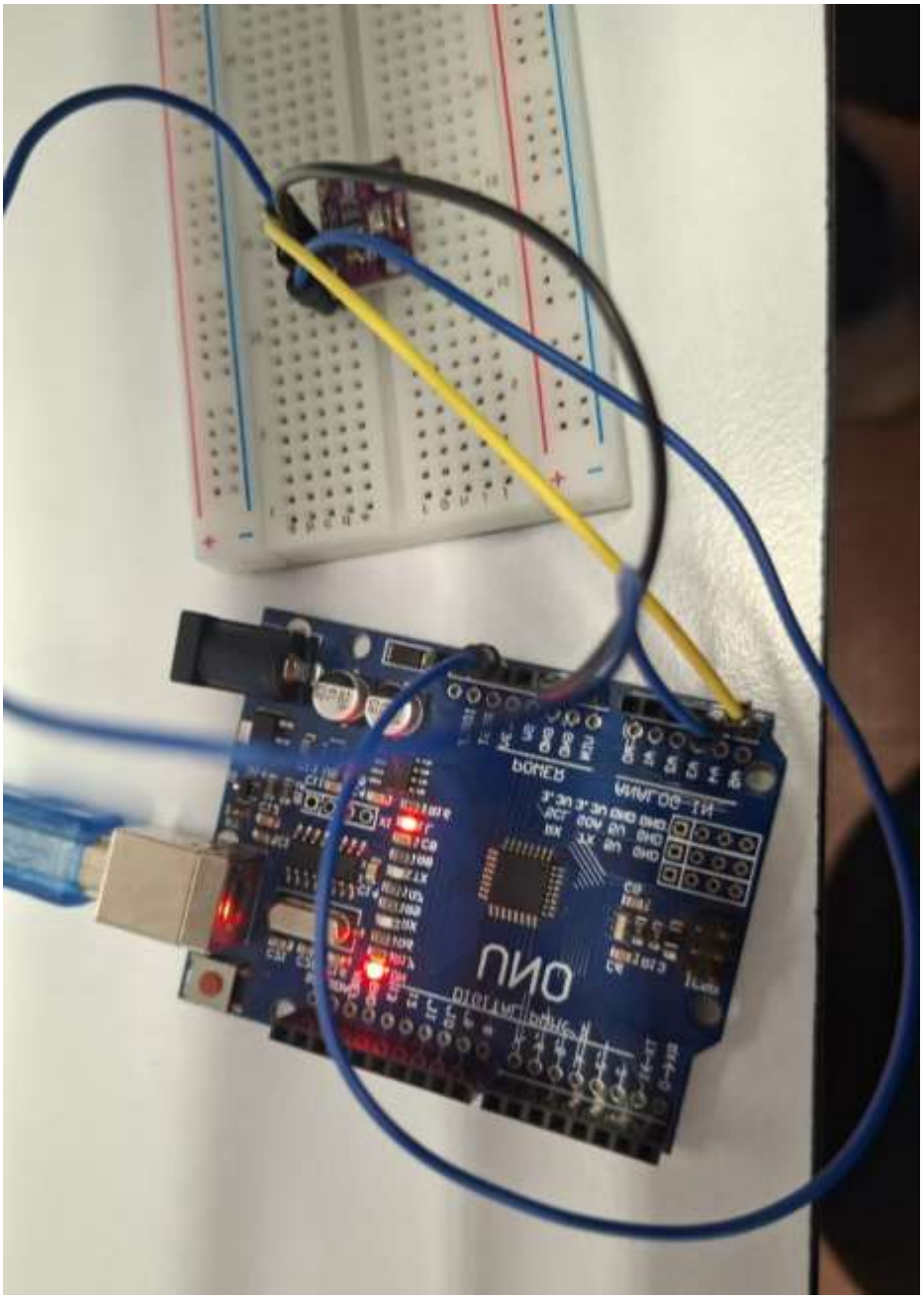

Imagen 7.
Circuito I2C armado en protoboard: Arduino UNO con el sensor BMP280 conectado a través de las líneas SDA y SCL. El LED de alimentación del Arduino está encendido (rojo), confirmando que el circuito recibe energía correctamente.

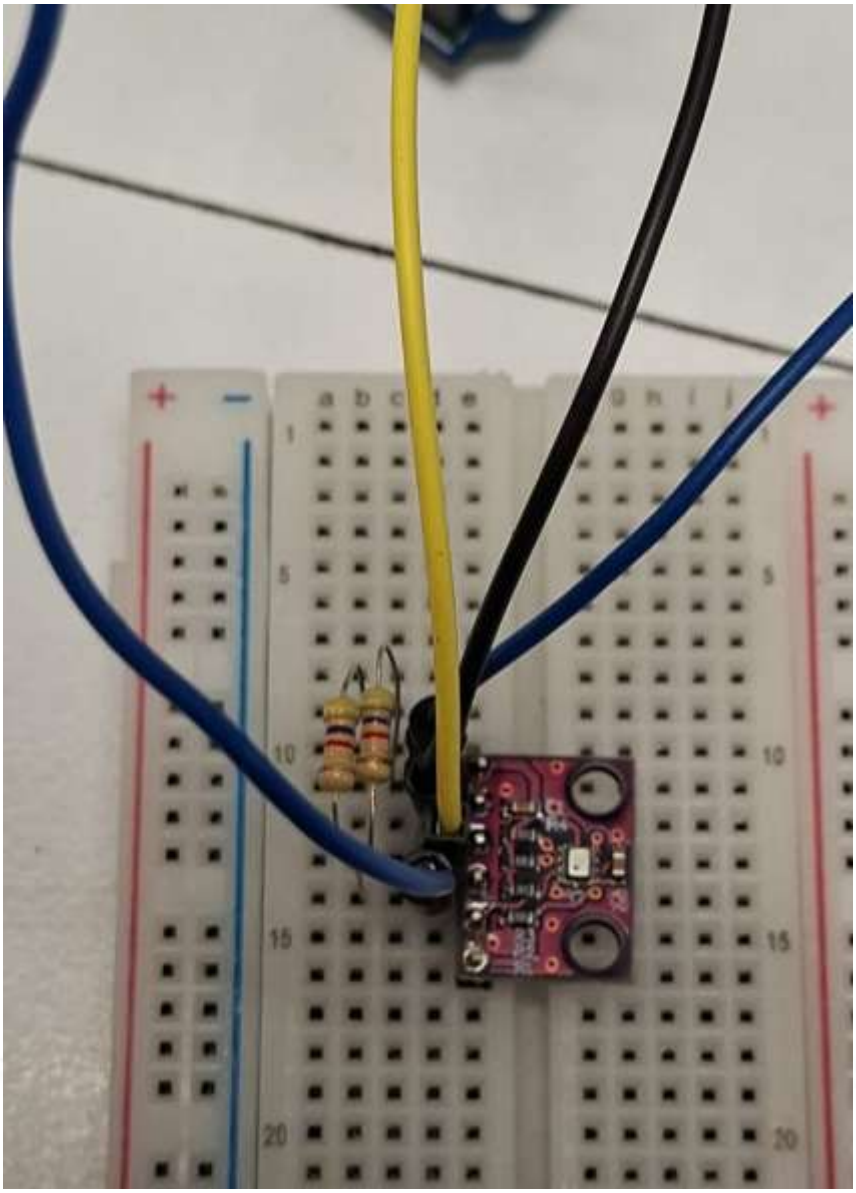

Imagen 8.
Detalle del módulo BMP280 insertado en la protoboard con las resistencias pull-up en las líneas SDA y SCL. Los cables de colores (amarillo, negro y azul) corresponden a las conexiones VCC, GND y datos del bus I2C respectivamente.

Preguntas de reflexión y reporte.

1. ¿En qué dirección I2C fue detectado el sensor? ¿Que determina si es 0x76 o 0x77?

El sensor fue detectado en la dirección I2C 0x76, aunque este valor puede variar dependiendo de la configuración física del dispositivo. La dirección I2C está determinada por el estado del pin SDO del sensor, el cual permite seleccionar entre dos direcciones posibles. Si el pin SDO se conecta a tierra (GND), la dirección será 0x76; mientras que si se conecta a voltaje (VCC), la dirección será 0x77. Esta característica permite utilizar múltiples sensores iguales en el mismo bus sin generar conflictos de direccionamiento.

2. ¿Cuántos dispositivos podrían conectarse simultáneamente en el mismo bus I2C del Arduino? ¿Qué limitación existe si dos sensores diferentes tienen la misma dirección fija?

En teoría, el bus I2C permite conectar hasta 127 dispositivos diferentes debido a su sistema de direccionamiento de 7 bits. Sin embargo, en la práctica este número es menor debido a factores físicos como la capacitancia del bus, la longitud de los cables y la estabilidad de la señal. Una limitación importante ocurre cuando dos dispositivos tienen la misma dirección I2C fija, ya que esto genera un conflicto de comunicación, impidiendo que el microcontrolador distinga entre ambos. En estos casos, es necesario cambiar la dirección de uno de los dispositivos (si es posible) o utilizar un multiplexor I2C para gestionar múltiples dispositivos con la misma dirección.

3. ¿Por qué se usa 'millis()' en lugar de 'delay(3000)' para temporizar las lecturas? Describe una situación donde delay() causaría un problema real.

Se utiliza la función millis() en lugar de delay(3000) porque permite medir el tiempo sin detener la ejecución del programa. El uso de delay() bloquea completamente el microcontrolador durante el tiempo especificado, impidiendo que se realicen otras tareas como la lectura de sensores o la recepción de datos por comunicación serial. En cambio, millis() permite implementar temporización sin bloquear el flujo del programa, lo que facilita la multitarea. Un problema real con delay() ocurriría en un sistema que necesita responder a comandos en tiempo real, ya que durante el tiempo de espera el microcontrolador no podría procesar nuevas entradas, provocando retrasos o pérdida de información.

4. La altitud calculada por el sensor, ¿coincide con la altitud real de tu ubicación? ¿Por qué podría haber diferencia? ¿Qué factores afectan la precisión?

La altitud calculada por el sensor generalmente no coincide exactamente con la altitud real de la ubicación. Esto se debe a que el cálculo se basa en la presión atmosférica, la cual puede variar dependiendo de las condiciones ambientales. Entre los factores que afectan la precisión se encuentran los cambios en el clima, el uso de un valor estándar de presión al nivel del mar (1013.25 hPa), la falta de calibración específica para la ubicación y las variaciones de temperatura. Por lo tanto, la altitud obtenida debe considerarse como una estimación aproximada y no como una medición exacta.

5. Si quisieras agregar una pantalla OLED SSD1306 (también I2C, dirección 0x3C) al mismo bus, ¿necesitarías cambiar el cableado de SDA/SCL? Describe como conectarlas juntas.

No es necesario cambiar el cableado principal del bus I2C para agregar una pantalla OLED SSD1306, ya que este protocolo permite conectar múltiples dispositivos en paralelo. Tanto el sensor como la pantalla comparten las mismas líneas SDA y SCL, así como la alimentación (VCC) y tierra (GND). La condición para que funcionen correctamente es que cada dispositivo tenga una dirección I2C diferente, por ejemplo, el sensor BMP280 puede usar 0x76 o 0x77, mientras que la pantalla OLED comúnmente utiliza la dirección 0x3C. De esta forma, ambos dispositivos pueden comunicarse sin interferencias en el mismo

VERDAD, BELLEZA, PROBIDAD

Laboratorio 07. PWM y Control Analogico: Dimmer de LED y Control Servo.

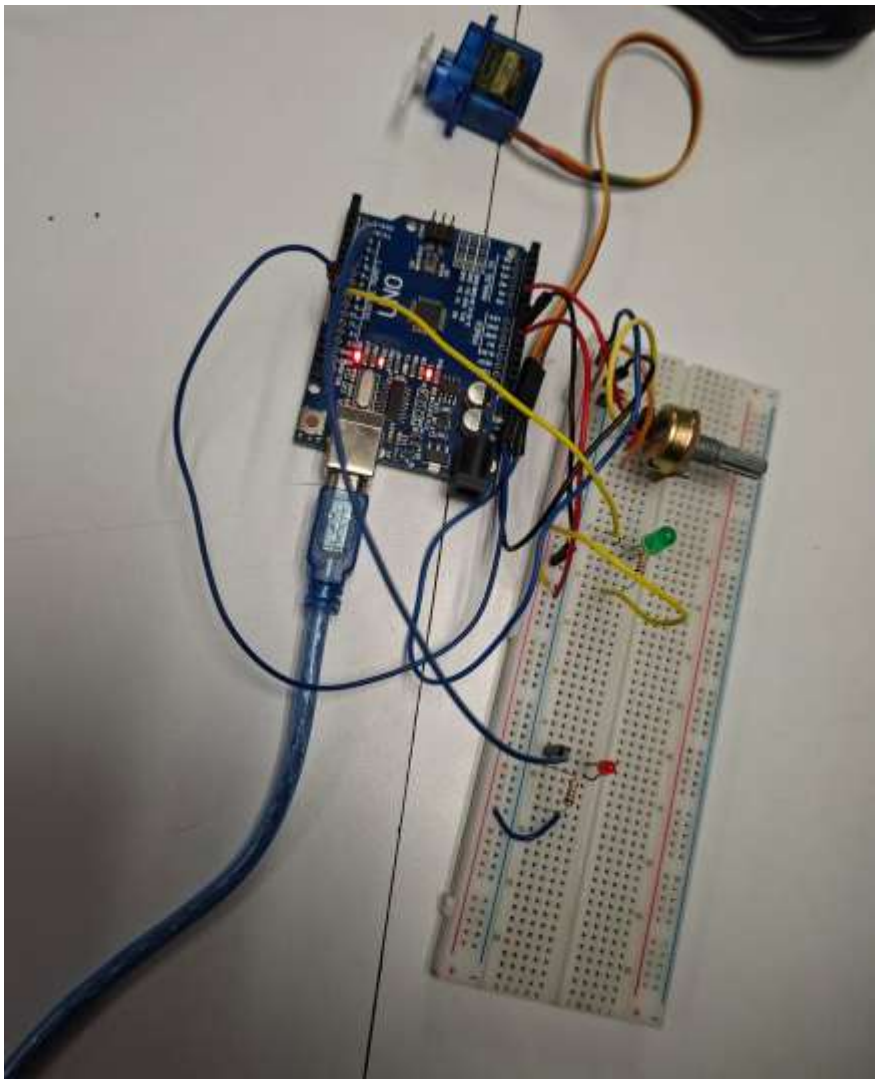

Imagen 9.

Circuito del Laboratorio 07: Arduino UNO conectado a un servo motor SG90 y un LED verde con potenciómetro en la protoboard. El sistema implementa control PWM donde el potenciómetro regula simultáneamente el brillo del LED (dimmer) y la posición angular del servo.

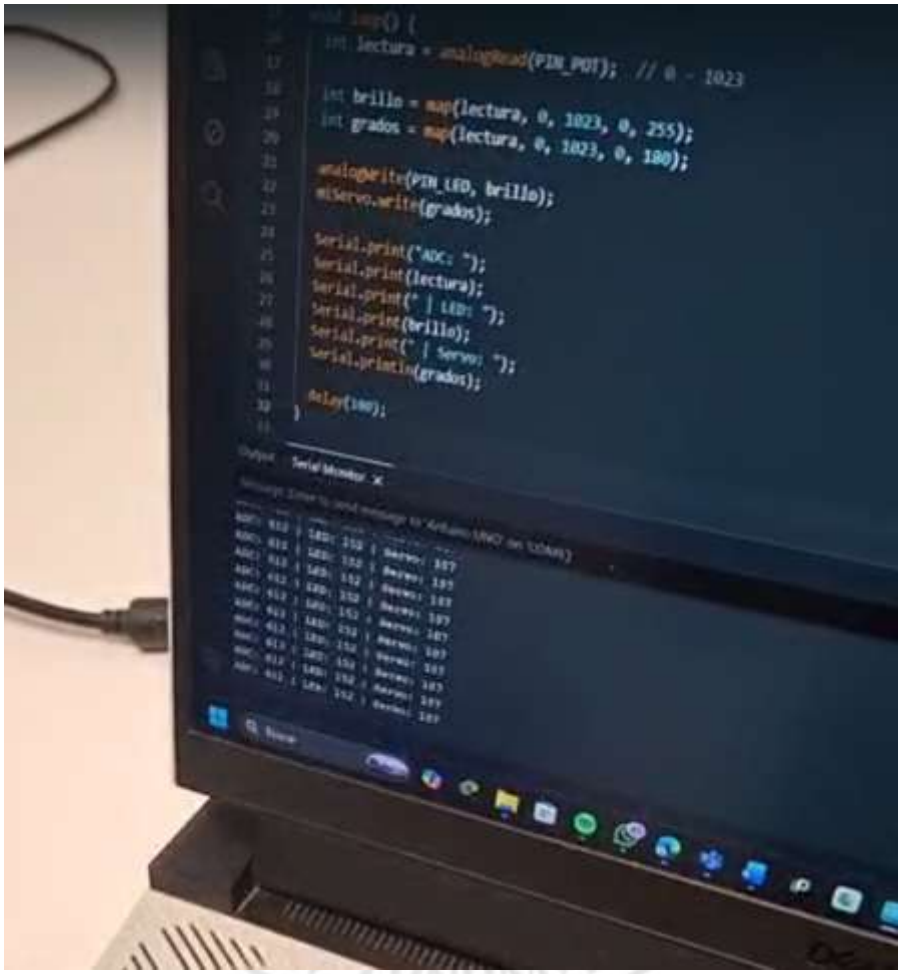

Imagen 10

Monitor Serial del Laboratorio 07 mostrando los valores en tiempo real: lectura ADC del potenciómetro (ADC ~612), nivel PWM del LED (LED: 152) y posición del servo en grados (Servo: 107). Se aprecia cómo la función map() convierte el rango ADC (0-1023) a los rangos correspondientes de brillo (0-255) y ángulo (0-180).

Preguntas de reflexión y reporte.

1. ¿Cuántos niveles de brillo distintos puede generar analogWrite() en Arduino UNO? ¿Cuántos bits tiene el DAC implícito del PWM?

Puede generar 256 niveles distintos (de 0 a 255). El "DAC implícito" (que en realidad es un periférico PWM) tiene una resolución de 8 bits ($2^8 = 256$).

2. Si el servo recibe un pulso de 1.5 ms cada 20 ms (50 Hz), ¿cuál es el duty cycle en porcentaje? ¿Y para un pulso de 1 ms?

La fórmula es:

$Duty Cycle = (\text{Ancho del pulso} / \text{Periodo total}) \times 100$

Para 1.5 ms:

$(1.5ms / 20ms) \times 100 =$ 7.5% (Posición central, aprox. 90°).

Para 1.0 ms:

$(1.0ms / 20ms) \times 100 =$ 5% (Posición inicial, 0°).

3. ¿Por qué el LED parece aumentar de brillo de forma no lineal cuando el duty cycle va de 0 a 255 de manera lineal? ¿Qué cambio en el código produciría una rampa de brillo más uniforme perceptualmente?

Es por la percepción logarítmica del ojo humano (Ley de Weber-Fechner); somos más sensibles a cambios de luz en entornos oscuros que en brillantes. Un cambio lineal de potencia no se traduce en un cambio lineal de brillo percibido.

El cambio de código se debería implementar una corrección Gamma o una tabla de búsqueda (Lookup Table) con una curva exponencial, donde los pasos iniciales crezcan más lentamente que los finales.

4. El ADC del ATmega328P es de 10 bits. ¿Cuántos niveles distintos puede diferenciar? Si el rango es de 0 a 5V, ¿cuál es la resolución en mV por nivel?

Puede diferenciar 1024 niveles (2^{10}). Tiene una resolución de $5V / 1024 \approx 0.00488V$, lo que equivale a 4.88 mV por cada unidad de incremento en la lectura analógica.

5. ¿Cómo modificarías el sketch para que al recibir el comando 'SCAN', el servo haga un barrido de 0 a 180 y de regreso, mientras reporta la posición cada 10 grados vía serial?

Se necesita agregar una lectura del puerto serial (Serial.available()) y una estructura de control para el barrido.

C++

```
if (Serial.available() > 0) {
  String comando = Serial.readStringUntil('\n');
  if (comando == "SCAN") {
    // Barrido de 0 a 180
    for (int pos = 0; pos <= 180; pos += 10) {
      miServo.write(pos);
      Serial.print("Posicion: "); Serial.println(pos);
      delay(200); // Tiempo para que el servo llegue
    }
    // Barrido de regreso
    for (int pos = 180; pos >= 0; pos -= 10) {
      miServo.write(pos);
      Serial.print("Posicion: "); Serial.println(pos);
      delay(200);
    }
```

```
  }
}
```

Laboratorio 08. SPI en Acción: Comunicación con pantalla TFT ILI9341.

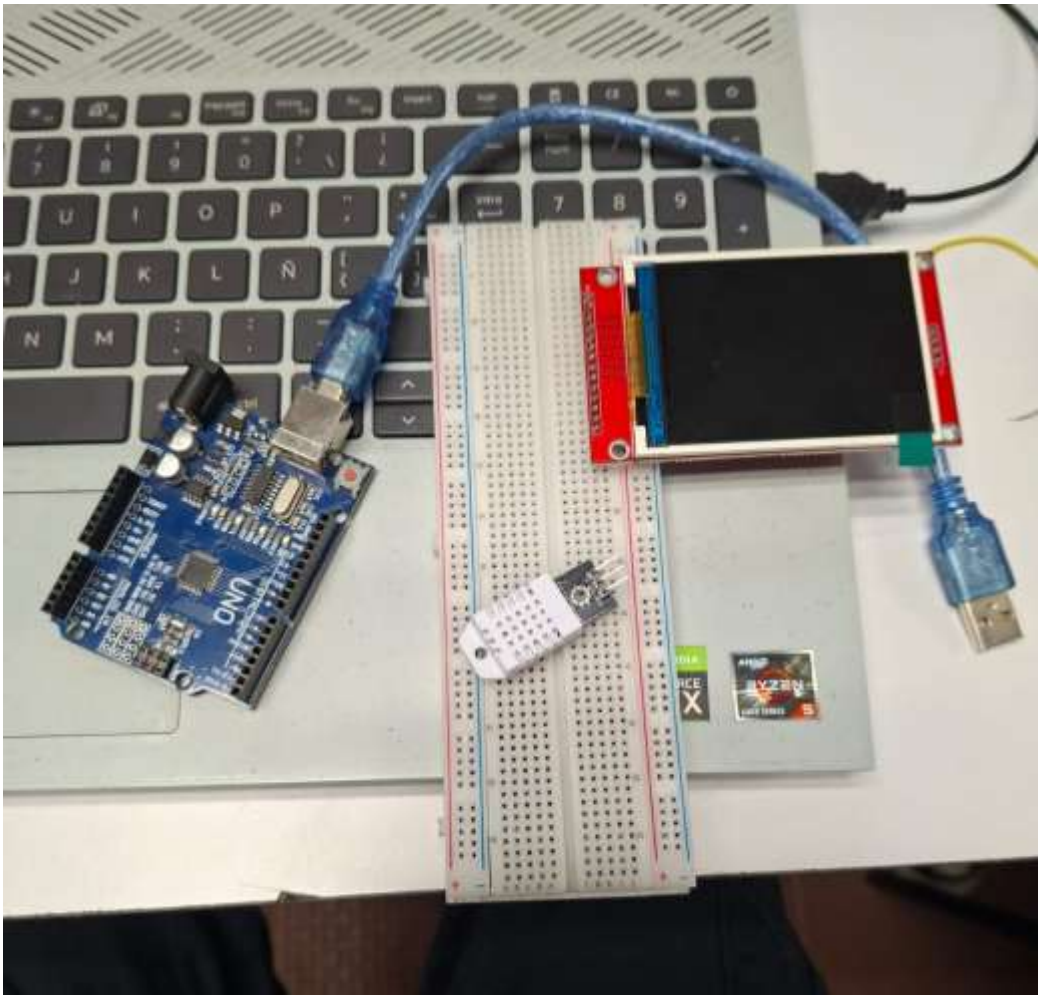

Imagen 11
Componentes del Laboratorio 08: Arduino UNO, pantalla TFT ILI9341 de 2.4 pulgadas con interfaz SPI y sensor DHT22, todos montados sobre una protoboard. El cable USB azul proporciona alimentación y comunicación con la computadora para la programación del microcontrolador.

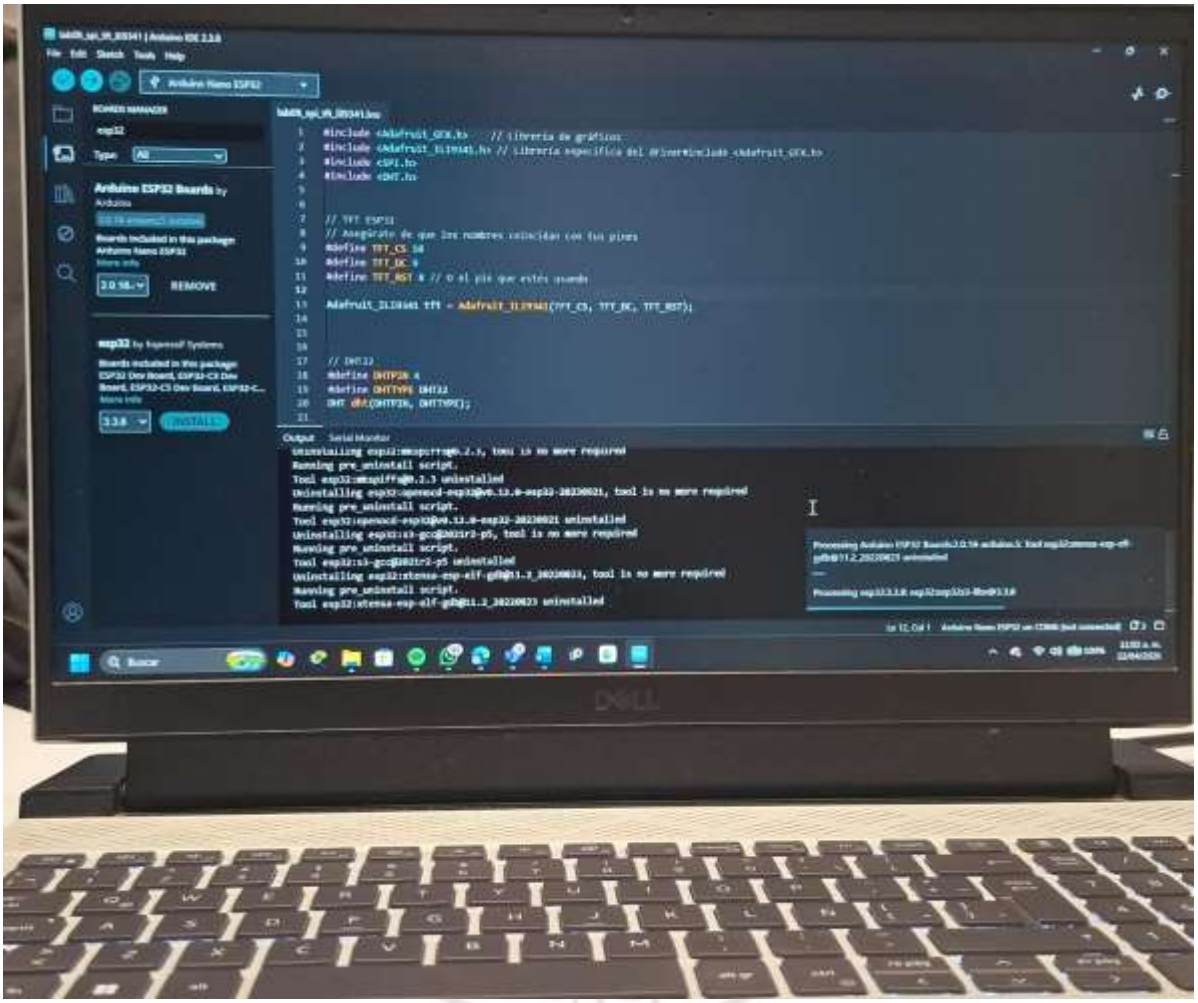

Imagen 12

Arduino IDE mostrando el proceso de instalación del soporte para ESP32 (Espressif Systems) mediante el Gestor de Placas. En pantalla se visualiza el código fuente del sketch SPI con la inclusión de las librerías Adafruit_GFX y Adafruit_ILI9341, así como la definición de los pines TFT_CS, TFT_DC y TFT_RST.

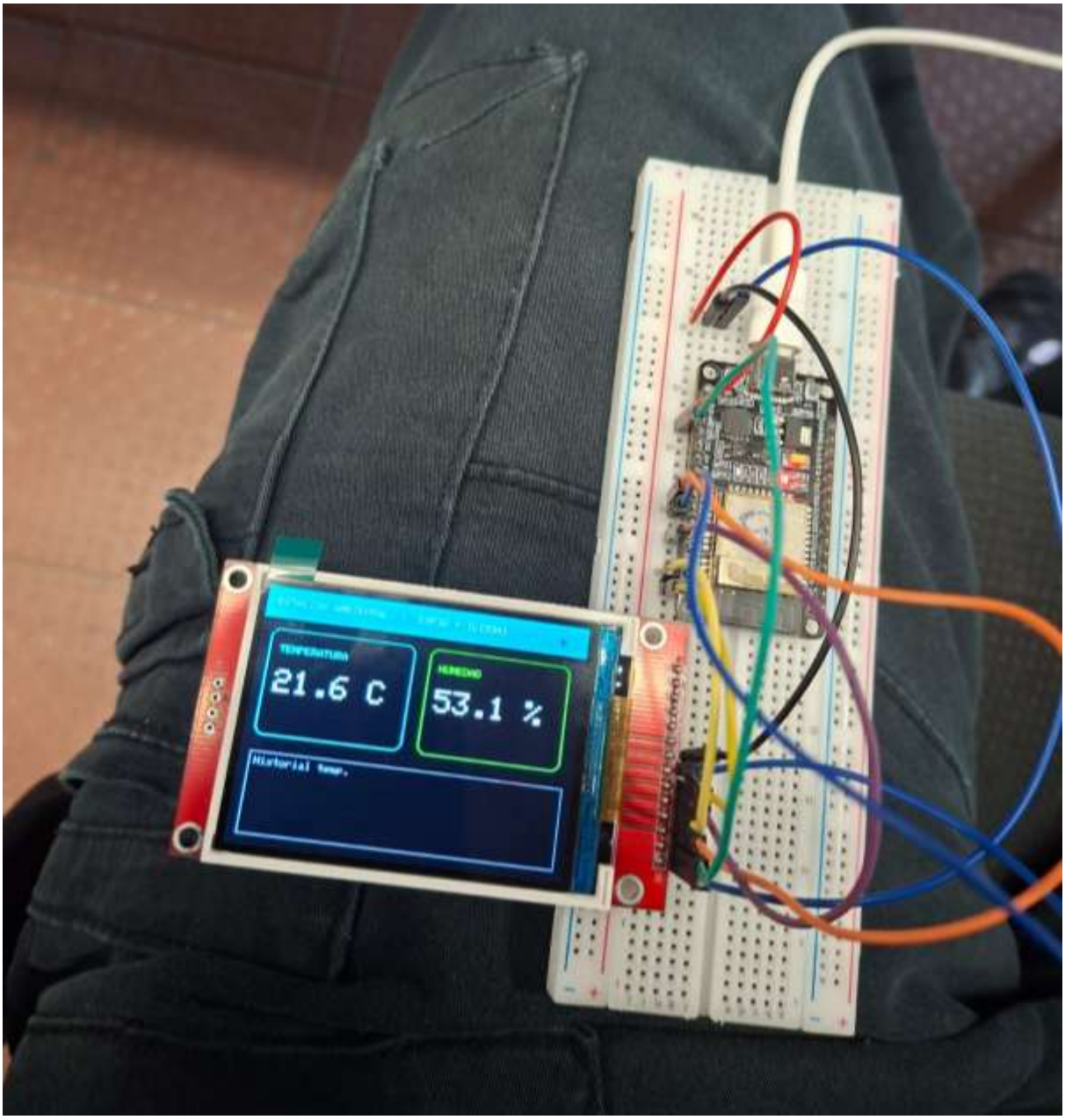

Imagen 13

Pantalla TFT ILI9341 en funcionamiento, mostrando la interfaz gráfica diseñada: temperatura de 21.6°C y humedad del 53.1% en recuadros con colores diferenciados, además de una sección de historial de temperatura en la parte inferior. El ESP32 (en la protoboard) gestiona la comunicación SPI a 40 MHz con la pantalla.

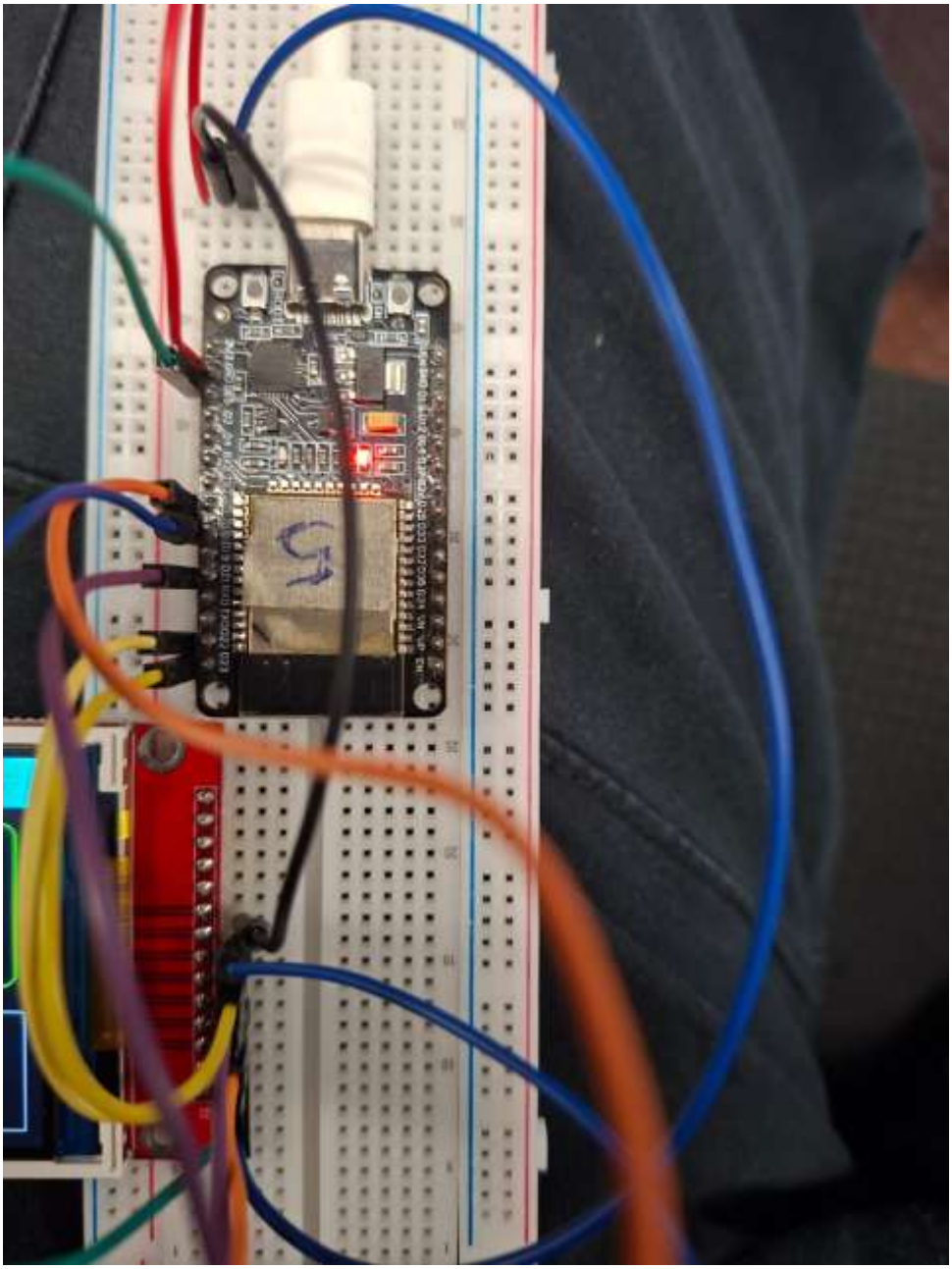

Imagen 14

Vista detallada del módulo ESP32 (NodeMCU) montado en la protoboard con su LED de actividad encendido. Los múltiples cables de colores corresponden a las señales del bus SPI: MOSI, MISO, SCK, CS, DC y RST, que interconectan el microcontrolador con la pantalla TFT ILI9341.

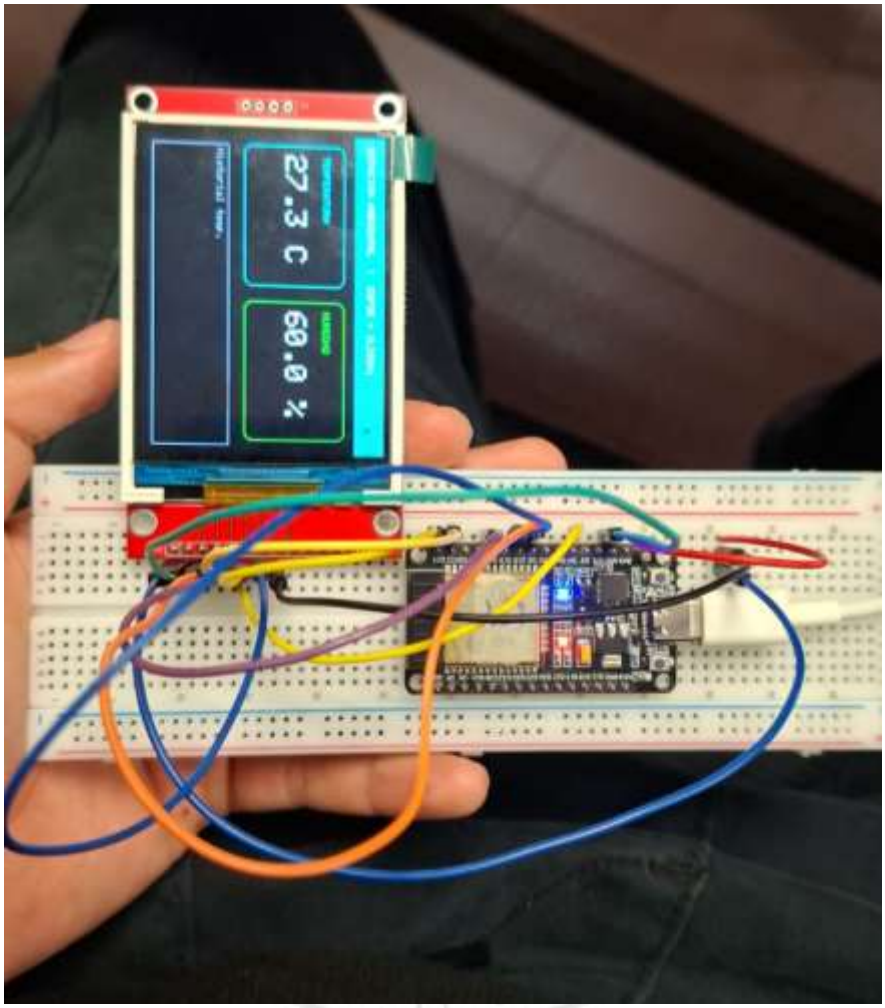

Imagen 15

Demostración final del Laboratorio 08: la pantalla TFT ILI9341 sostenida en mano muestra los datos del sensor DHT22 en tiempo real (temperatura ~23°C y humedad ~69%) con una presentación visual clara. Se aprecia el circuito completo con el ESP32, la protoboard y todos los cables SPI correctamente conectados.

Preguntas de reflexión y reporte.

1. ¿Cuántos bytes de datos se necesitan para actualizar un solo pixel en la pantalla ILI9341? ¿Y para actualizar toda la pantalla (240x320)?

Cada pixel de la pantalla ILI9341 requiere 2 bytes de datos, debido al formato de color RGB565. Considerando que la pantalla tiene una resolución de 240 × 320 píxeles, el total de datos necesarios para actualizar completamente la pantalla es de 153,600 bytes. Esta cantidad representa el volumen de información que debe transmitirse para dibujar un frame completo.

2. A 40 MHz de SPI, ¿cuánto tiempo mínimo (teórico) se necesita para enviar todos los datos de un frame completo? ¿Por qué en la práctica es mayor?

A una velocidad de 40 MHz en SPI, el tiempo mínimo teórico para enviar todos los datos de un frame completo es aproximadamente 30.7 milisegundos. Sin embargo, en la práctica este tiempo es mayor debido a factores como la sobrecarga del software, la ejecución de funciones de la biblioteca gráfica, la latencia del microcontrolador y las limitaciones del hardware. Estos elementos introducen retardos adicionales que reducen el rendimiento real respecto al teórico.

3. ¿Por qué se usa la función fillRect() para borrar el área del número antes de redibujar el nuevo valor, en lugar de usar fillScreen() completo?

Se utiliza la función fillRect() porque permite actualizar únicamente una sección específica de la pantalla en lugar de toda la superficie. Esto reduce significativamente la cantidad de datos que se deben enviar por SPI, mejora el rendimiento y evita efectos visuales como el parpadeo. En contraste, fillScreen() borra y redibuja toda la pantalla, lo que resulta innecesario cuando solo cambian ciertos elementos, como valores numéricos.

4. Compara la velocidad de actualización de la pantalla cuando el ESP32 opera el SPI a 10 MHz vs 40 MHz. ¿Qué diferencia se observa visualmente?

Al operar el SPI a 40 MHz, la pantalla presenta una actualización mucho más rápida, lo que se traduce en animaciones más fluidas y menor tiempo de respuesta visual. En cambio, a 10 MHz se observa una actualización más lenta, con posibles retrasos y menor suavidad en los cambios de imagen. Esta diferencia es especialmente evidente en aplicaciones que requieren actualizaciones frecuentes o gráficos dinámicos.

VERDAD, BELLEZA, PROBIDAD

5. ¿Cuáles son las ventajas de usar SPI sobre I2C para una pantalla grafica de alta resolución? ¿Existiría alguna ventaja de usar I2C para este caso?

El protocolo SPI ofrece ventajas importantes para pantallas gráficas de alta resolución, como una mayor velocidad de transferencia de datos, comunicación full-duplex y mejor rendimiento en el manejo de grandes volúmenes de información. Estas características lo hacen ideal para dispositivos como la pantalla ILI9341. Por otro lado, I2C tiene la ventaja de requerir solo dos líneas de comunicación, lo que simplifica el cableado, pero su velocidad es considerablemente menor. Por esta razón, I2C no es adecuado para pantallas de alta resolución, aunque sí puede ser útil en dispositivos más simples o con menor demanda de datos.